

OASIS

Mode d'emploi POC IoT — pas à pas, illustré

Phase A : logiciel (lançable maintenant) · Phase B : matériel

Document interne · transparence totale · 12 juin 2026

Les schémas sont **dessinés** (pas des photos) ; le pinout est à vérifier contre tes cartes.

0. À lire avant de commencer

Deux vérités à accepter :

- **Phase A (logiciel) marche aujourd'hui** et prouve la *logique* du nœud (crypto, mesh, airtime, duty-cycle). Aucun matériel requis.
- **Phase B (matériel) n'est PAS encore exécutable** : le driver radio `SX1262` est un **stub**. Il faut d'abord **l'écrire** (~2–3 j, via `sx126x-rs / lora-phy`) **et** acheter le matériel (voir BOM). La Phase B ci-dessous est le **protocole à suivre une fois ces deux conditions remplies**.

Les illustrations sont des **schémas** (pas des photos d'un vrai banc). Les numéros de broches et commandes sont issus des datasheets/wikis — **vérifie-les contre tes cartes réelles** avant de brancher.

1. Vue d'ensemble

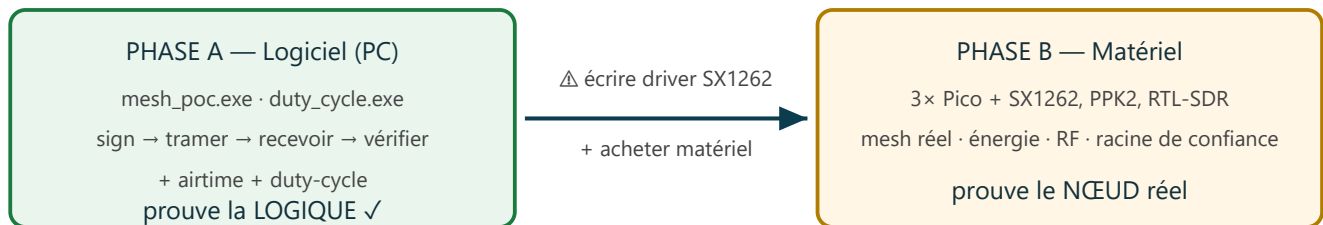


Fig. 1 — Deux phases. La A est lançable ; la B est conditionnée au driver + matériel.

PHASE A — POC logiciel (maintenant, sans matériel)

A1. Prérequis

- Rust stable installé (rustup), `cargo --version` ≥ 1.80 .
- Le dépôt OASIS (c:\dev\oasis).

Critère : `cargo --version` répond sans erreur.

A2. Lancer le POC mesh end-to-end

```
cd c:\dev\oasis\oasis-lora-transport
cargo run --release --features std --example mesh_poc
```

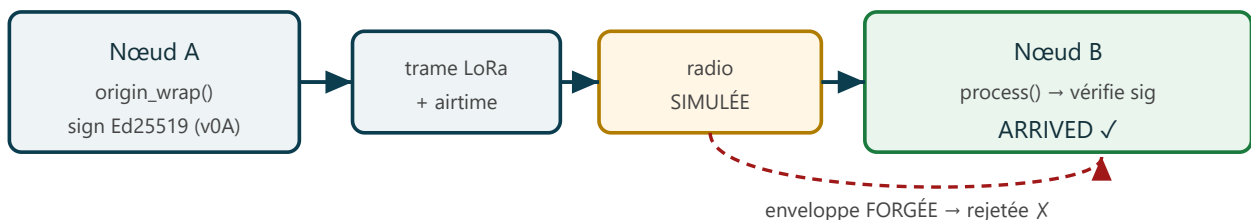


Fig. 2 — Chaîne validée : signature → trame → radio simulée → réception → vérification ; la forge est rejetée.

Sortie attendue (réelle) :

```
[1] 2 noeuds Ed25519 (A=0xAA, B=0xBB), pubkeys croisees enregistrees.
[2] Lien LoRa simule (SF7, BW code 4).
[3] A signe + emet (28 o charge -> 117 o enveloppe, airtime 210.2 ms).
[4] B recoit + VERIFIE la signature: ARRIVED (msg_id=0x100ba66a4ed81db5, hops=0).
[5] Securite: enveloppe FORGEE rejetee (bad mesh signature). OK.
[6] EU868 1%: 7/50 envois legaux a 1 msg/s (le reste throttle).
```

Critères de succès (cocher) : ☐ ligne [4] contient **ARRIVED** + `msg_id=` (signature Ed25519 vérifiée, sur lien **simulé**) · ☐ ligne [5] = **FORGEE rejetee (bad mesh signature)** · ☐ ligne [6] = **7/50** (test interne du throttler : 50 essais à 1/s depuis t=0). Si [4] dit « rejette le message legitime » ⇒ régression crypto à investiguer.

Échecs courants : oubli de `--features std` (l'exemple ne compile pas) ; ligne [4] = « rejette le message legitime » ⇒ régression crypto à investiguer.

A3. Démo airtime / duty-cycle

```
cargo run --release --example duty_cycle
```

Affiche le temps-on-air par SF (SF7 $\approx$ 61,7 ms, SF12 $\approx$ 1 482,75 ms pour 24 o) et le pacing EU868 1 %.

Critère : SF12 > SF7 (longue portée = plus lent) ; throttling visible ($\ll$ 150 envois acceptés).

Fin de Phase A. Si A2 et A3 passent, la **logique** du nœud est prouvée. Tu peux passer commande du matériel en confiance. **Ne confonds pas ça avec « validé IoT » — la radio reste à prouver (Phase B).**

PHASE B — POC matériel (après driver + achat)

Conditions de démarrage de la Phase B : (1) driver `Sx1262Driver` réel écrit et testé en boucle locale ; (2) matériel reçu (3× Pico + 3× hat SX1262 868 MHz, sonde SWD, PPK2, RTL-SDR — cf. BOM). Sans (1), les radios ne transmettent rien.

⚠ **INSTRUCTIONS NON TESTÉES — propositions schématiques.** Aucun pas B1–B7 n'a été exécuté par l'auteur ; tout vient des datasheets/wikis. Le driver SX1262 est un **stub bloquant** (`oasis-lora-transport/src/sx1262.rs:78` renvoie « init not yet implemented »). Vérifie chaque broche et chaque commande contre TON matériel avant de t'y fier.

Légal & réglementaire (avant toute émission) : en bande ISM 868 MHz (EU) l'émission est réglementée — respecter le **1 % de duty-cycle** (ou FHSS), les limites d'**EIRP** (typiquement $\leq +14$ dBm), et ne pas émettre en zone interdite. Aux US : bande **915 MHz**, contraintes différentes (masque spectral, dwell time). **Émettre hors règles est illégal.**

B1. Flasher un Raspberry Pi Pico



Fig. 3 — Maintenir BOOTSEL en branchant l'USB → un lecteur « RPI-RP2 » apparaît → y déposer le `.uf2`.

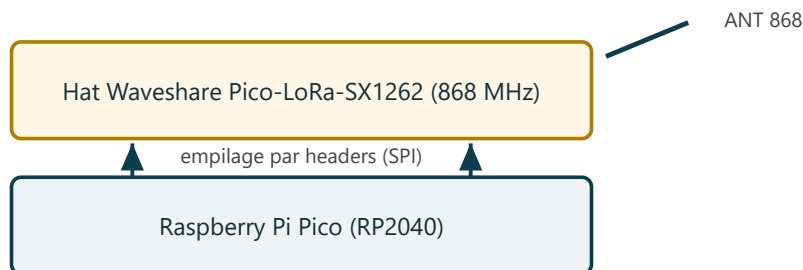
Générer le firmware (depuis le crate RP2040) :

```
# installer une fois : outil UF2 + la cible MCU
cargo install elf2uf2-rs
rustup target add thumbv6m-none-eabi
cd c:\dev\oasis\oasis-mcu-demo
cargo build --release          # cible thumbv6m auto-sélectionnée (.cargo/config.toml)
# le firmware = le binaire du crate (src/main.rs) ; les autres [[bin]] sont des bench
elf2uf2-rs target/thumbv6m-none-eabi/release/oasis-mcu-demo firmware.uf2
```

Critère : après dépôt du `.uf2`, le Pico reboote et émet sur l'UART/USB-série les logs du noyau (les mêmes qu'en simulateur Wokwi).

À vérifier : nom exact du binaire (`ls target/thumbv6m-none-eabi/release/`) ; pour le debug/mesure RAM, brancher la sonde SWD (SWCLK/SWDIO).

B2. Monter le SX1262 + antenne



SPI / contrôle (câblage canonique sx1262.rs — vérifier le wiki de TA carte) :

SCK→GP10 · MOSI→GP11 · MISO→GP12 · NSS/CS→GP13 · BUSY→GP14 · DIO1→GP15 · RESET→GP16 · 3V3 · GND

Fig. 4 — Le hat s'empile sur le Pico (bus SPI). Brancher l'antenne 868 MHz **avant** toute émission.

Ne JAMAIS émettre sans antenne (risque de dommage de l'étage PA). Les broches ci-dessus suivent le **câblage canonique de oasis-rt/src/sx1262.rs** ; **confirmez-les sur le wiki exact de ta carte Waveshare** (et au multimètre) avant de brancher, puis passez-les au driver `Sx1262Driver`.

B3. Lien point-à-point (2 nœuds)

⚠ **Antenne 868 connectée · MÊMES fréquence/SF/BW sur les 2 nœuds · vérifier le pinout au multimètre (aucun court 3V3↔GND) AVANT d'alimenter.**

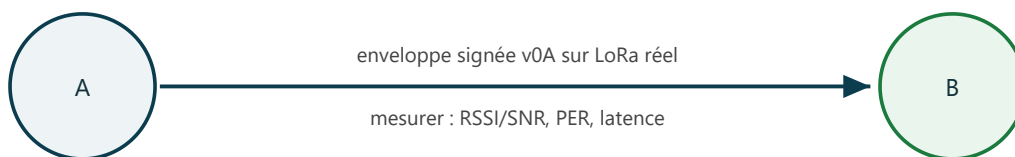


Fig. 5 — A émet, B reçoit et vérifie. Test d'abord à 1 m (sanity), puis en portée croissante.

Critère : B affiche **ARRIVED** (signature OK) avec RSSI/SNR plausibles ; refaire à distance croissante et tracer PER vs distance.

B4. Mesh multi-saut (3 nœuds) — le test clé

⚠ **Antennes 868 sur les 3 nœuds · mêmes fréquence/SF/BW partout.**

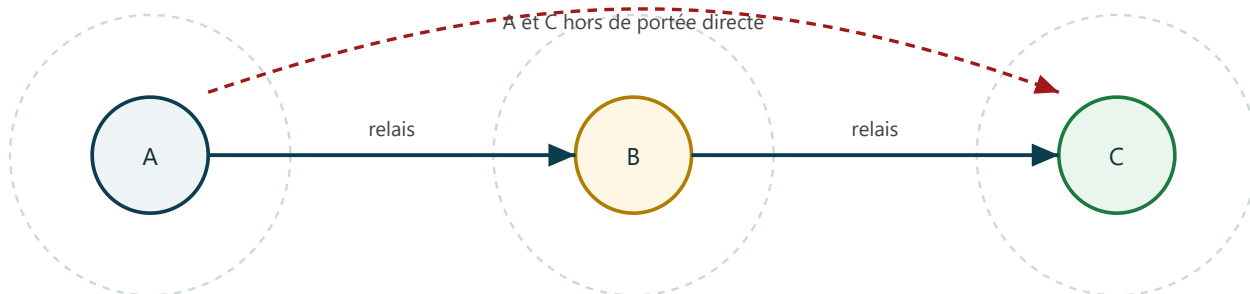


Fig. 6 — Placer B entre A et C, A et C hors portée directe. Un message d'A atteint C **uniquement si B relaie** (TTL/forward de `mesh.rs`).

Critère (cœur du POC) : C reçoit le message d'A **via B** (hops ≥ 1) ; couper B $\Rightarrow$ C ne reçoit plus. Vérifier la **dédup** (pas de tempête de rebroadcast).

B5. Conformité RF (RTL-SDR)

⚠ **Antenne sur le nœud émetteur · le RTL-SDR écoute (ne transmet pas).**



vérifier : fréquence,
airtime $\approx$ calc., 1 % duty,
saut FHSS

Fig. 7 — Le RTL-SDR « voit » les chirps LoRa : confirmer la fréquence, comparer l'airtime mesuré au calcul (`airtime.rs`), et vérifier que le 1 % est respecté.

Critère : airtime observé $\approx$ valeur calculée ($\pm 10\%$) ; pas d'émission au-delà du budget 1 % ; sauts FHSS visibles si activés.

B6. Mesure d'énergie (PPK2) — le juge de paix IoT

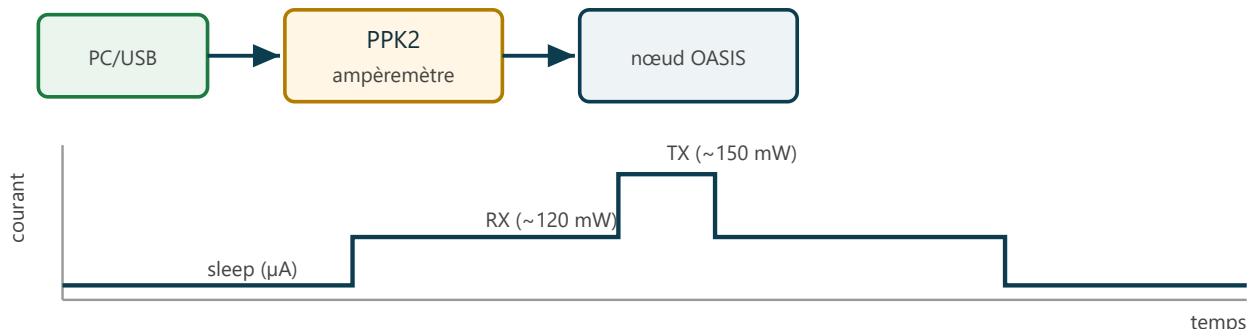


Fig. 8 — PPK2 en série entre l'alim et le nœud. Profil attendu : sommeil très bas, RX moyen, pic à l'émission. C'est ce qui dit si le nœud peut vivre sur batterie.

Critère (ÉCHEC ATTENDU aujourd'hui) : mesurer RX (~ 120 mW) et TX (~ 150 mW). **Le régime « sleep » sera ABSENT** — le code n'a pas encore de gestion d'énergie. Le but de ce test est justement de **chiffrer ce trou** et de servir de référence avant le travail énergie (Phase 1).

B7. [DIFFÉRÉ — Phase 2] Racine de confiance (ATECC608B, I2C)

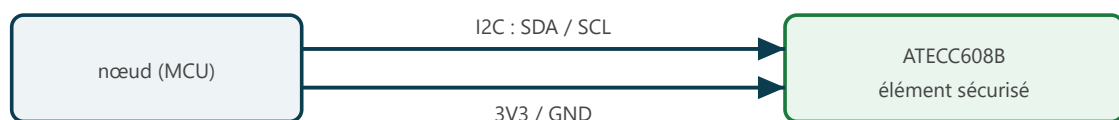


Fig. 9 — L'élément sécurisé sur le bus I2C : génère/stocke la clé privée du nœud **hors RAM**.

Critère : la clé d'identité du nœud vit dans l'ATECC608B (jamais exposée en RAM) ; `oasis-secure-element` (aujourd'hui stub) implémente le séquençement I2C réel.

Récapitulatif des critères de succès

Étape	Prouve	Critère de réussite
A2 mesh_poc	logique crypto+mesh	ARRIVED + forge rejetée
A3 duty_cycle	airtime + conformité (calc.)	throttling visible
B1 flash	boot silicium réel	logs noyau sur série
B3 2 nœuds	lien LoRa réel	ARRIVED + RSSI/SNR + PER
B4 3 nœuds	mesh multi-saut	C reçoit via B ; couper B ⇒ plus de réception
B5 RTL-SDR	conformité RF réelle	airtime mesuré ≈ calc. ; 1 % respecté
B6 PPK2	énergie	3 régimes mesurés ; autonomie estimée
B7 ATECC608B	racine de confiance	clé hors RAM

Dépannage rapide & rappel d'honnêteté

- **A2 ne compile pas** : ajouter `--features std`.
- **Pico non détecté** : maintenir BOOTSEL pendant le branchement ; vérifier le câble (data, pas charge seule).
- **Rien reçu en B3** : driver non écrit (stub), antenne absente, fréquences/SF différents entre nœuds, ou mauvais pinout SPI.
- **B6 montre une conso constante élevée** : attendu — pas de sleep dans le code aujourd'hui (à corriger).

Rappel : Phase A verte = logique prouvée, **pas** « IoT validé ». La preuve réelle est la Phase B sur silicium + radio. Le binaire logiciel est un **pré-vol**, pas un vol.

OASIS — Mode d'emploi POC IoT · schémas dessinés, à vérifier contre le matériel réel · 12 juin 2026